

Budgeted Embedding Table For Recommender Systems

Yunke Qu

The University of Queensland
Brisbane, Australia
yunke.qu@uq.net.au

Quoc Viet Hung Nguyen

Griffith University
Gold Coast, Australia
henry.nguyen@griffith.edu.au

Tong Chen

The University of Queensland
Brisbane, Australia
tong.chen@uq.edu.au

Hongzhi Yin*

The University of Queensland
Brisbane, Australia
h.yin1@uq.edu.au

ABSTRACT

At the heart of contemporary recommender systems (RSs) are latent factor models that provide quality recommendation experience to users. These models use embedding vectors, which are typically of a uniform and fixed size, to represent users and items. As the number of users and items continues to grow, this design becomes inefficient and hard to scale. Recent lightweight embedding methods have enabled different users and items to have diverse embedding sizes, but are commonly subject to two major drawbacks. Firstly, they limit the embedding size search to optimizing a heuristic balancing the recommendation quality and the memory complexity, where the trade-off coefficient needs to be manually tuned for every memory budget requested. The implicitly enforced memory complexity term can even fail to cap the parameter usage, making the resultant embedding table fail to meet the memory budget strictly. Secondly, most solutions, especially reinforcement learning based ones derive and optimize the embedding size for each user/item on an instance-by-instance basis, which impedes the search efficiency. In this paper, we propose Budgeted Embedding Table (BET), a novel method that generates table-level actions (i.e., embedding sizes for all users and items) that is guaranteed to meet pre-specified memory budgets. Furthermore, by leveraging a set-based action formulation and engaging set representation learning, we present an innovative action search strategy powered by an action fitness predictor that efficiently evaluates each table-level action. Experiments have shown state-of-the-art performance on two real-world datasets when BET is paired with three popular recommender models under different memory budgets.

CCS CONCEPTS

• Information systems → Recommender systems.

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).
WSDM '24, March 4–8, 2024, Merida, Mexico

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0371-3/24/03...\$15.00
<https://doi.org/10.1145/3616855.3635778>

KEYWORDS

recommender systems, neural architecture search

ACM Reference Format:

Yunke Qu, Tong Chen, Quoc Viet Hung Nguyen, and Hongzhi Yin. 2024. Budgeted Embedding Table For Recommender Systems. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM '24)*, March 4–8, 2024, Merida, Mexico. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3616855.3635778>

1 INTRODUCTION

Recommender systems (RSs) predict a user's preference for an item according to observed user-item interactions [51, 61] and have been indispensable across numerous modern platforms such as e-commerce and social media [23, 27, 64]. In contexts where computations on local devices are essential, such as federated recommendation [39] and Internet of Things (IoT) services [16], the need for RSs with small memory footprints becomes critical due to highly constrained computational resources. The main bottleneck for downsizing RSs is often the embedding table, which represents users and items with distinct embedding vectors to facilitate pairwise similarity computation. As such, the size of the embedding table can quickly exhaust the available memory, posing a significant challenge to achieving scalable and resource-friendly RSs.

Such memory bottleneck is attributed to conventional embedding tables, which typically allocate fixed and uniform embedding sizes to each user and item [8, 18]. Such a design may require an excessive number of parameters in industry-scale applications, e.g., the benchmark recommender in [11] has more than 25 billion parameters in its embedding table and requires 100 GB memory. In this regard, researchers have developed methods to achieve lightweight embeddings while maintaining its expressiveness. One stream of methods aim to prune redundant parameters of the embedding table. For example, [37] proposed OptEmbed, a novel embedding sparsification framework that prunes redundant embeddings by learning an importance score for each feature. These methods mostly optimize a loss function combining a recommendation term and a sparsity regularizer. Apart from sparsification/pruning methods, another line of works featuring reinforcement learning (RL) techniques have also emerged. Those methods adopt an RL-based policy network to search for the optimal embedding sizes for each user/item, where the embedding sizes (i.e., actions) can be selected from an action space consisting of either a predefined set of values [?] or a continuous range [41]. Similar to sparsification methods, the reward functions in RL-based lightweight embedding approaches

also bind the recommendation quality term with a memory complexity penalty. By tuning a trade-off coefficient among the two terms, they can adjust the regularization strength on the parameter consumption, thus achieving the desired memory efficiency.

Despite the possibility of balancing performance and embedding parameter size, these two types of methods are still away from practical use for two reasons. Firstly, such a desired balance is achieved by jointly optimizing an additional space complexity constraint on top of the recommendation objective. For example, in [18], this is formulated as the difference between the budgeted embedding parameter size and the actual parameter consumption of the compressed embedding table. However, such an implicit loss term means that the resultant embedding table may not necessarily meet the desired parameter budget every time [8]. Moreover, the optimization quality is highly sensitive to the trade-off coefficient, which needs to be heuristically tuned for each memory budget.

The second reason is that, to achieve optimal recommendation performance under each memory budget, the embedding size is decided on an instance-by-instance basis w.r.t. each user/item. Taking a recent RL-based method CIESS [41] as an example, a policy network predicts the optimal action for every single user and item, based on which the backbone RS is trained and evaluated to calculate the reward, enabling training the policy network. On the one hand, ideally, the policy network is expected to be iteratively updated w.r.t. each instance (i.e., user/item), but this can be prohibitively inefficient given the large number of users and items. On the other hand, performing only one update per batch/epoch leads to slower convergence due to the aggregated nature of the reward function, and may potentially result in inferior generalization [20].

In response to both challenges, we propose Budgeted Embedding Table (BET) for RSs, an automated input embedding size search paradigm that can efficiently generate size decisions and perform optimization at the embedding table level. An overview of BET is provided in Figure 1. To address the first challenge, we innovatively propose a bounded sampler that draws embedding sizes for all users/items from a collection of probabilistic distributions, with the guarantee that the total parameter usage is capped. As the embedding sizes for all instances in the embedding table are produced all at once, we term this a *table-level action*. This strategy frees us from the need for manually designing and tuning the implicit parameter constraint in the reward/loss function, and allows us to enforce the memory budget with a negligible cost.

To tackle the second challenge, we further pair the generated table-level actions with a fitness prediction network to evaluate the quality of those actions. Essentially, in each search iteration, after sampling several table-level actions, we need to identify the ones that promise high recommendation accuracy. A straightforward solution is to greedily evaluate each candidate action by training and evaluating a backbone RS from scratch with the suggested embedding sizes [18, 41, 62?], which apparently impedes efficiency.

One may think of a surrogate function [3, 8] that “predicts” the fitness of each action, instead of explicitly evaluating it. Accurately predicting the performance of the table-level actions is crucial yet another major challenge in this regard, as either overestimating or underestimating an action’s performance will eventually cause sub-optimal embedding size decisions. To align with the evaluation of table-level actions in BET, our fitness predictor innovatively utilizes

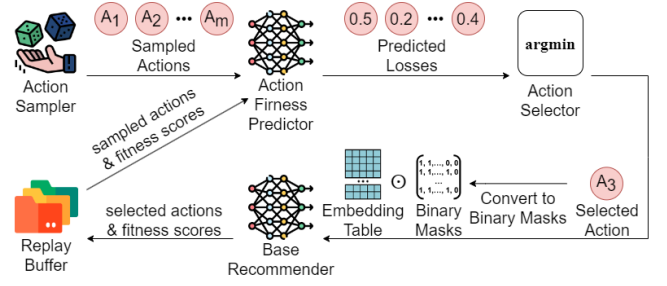


Figure 1: Overview of BET.

a set-based formulation of actions, and embeds each action with DeepSets [59], where the learned high-quality action embeddings can efficiently provide reliable performance feedback on each table-level action. Furthermore, the set-based action representations can be inductively learned, which is essential for our iterative search algorithm that constantly generates unseen actions that need to be evaluated.

To summarize, our work presents the contributions below:

- We identify the practicality bottlenecks of existing embedding size search methods for lightweight RSs, namely the implicit and non-scalable parameter size constraints for meeting different memory budgets, and the inefficient embedding size generation and evaluation for every instance.
- We propose BET, a novel table-level embedding size search approach featuring an efficient action sampling strategy that generates table-level actions for all users and items and a DeepSets-based fitness prediction network that accurately predicts their performance to reduce action evaluation costs.
- Extensive empirical comparisons have been carried out on two real-world datasets by pairing state-of-the-art embedding size search algorithms with various backbone RSs, which confirm the advantageous efficacy of BET.

2 PRELIMINARIES

To provide a detailed research background, we hereby discuss relevant research topics.

2.1 Deep Recommender Systems

Researchers have proposed numerous deep recommender models to capture the user-item relationships. The first line of deep recommender systems are based on MLPs. For instance, Neural Collaborative Filtering (NCF) [15] integrates a MLP with a matrix factorization component to learn the two-way interactions between users and items. AutoRec [45] uses an autoencoder to model the user-item interactions in a low-dimensional hidden space and perform collaborative filtering. Lin et al. proposed a novel sequential recommender system, Dual Contrastive Network [31] that generates self-supervised signals by considering both user and item perspectives and refining representation learning through dual representation contrastive learning. In addition to MLP-based methods, graph-based architectures [14, 53, 58] have also been studied. For example, Neural Graph Collaborative Filtering (NGCF) [53] extends traditional neural collaborative filtering by modeling user-item interactions with graph convolution networks. LightGCN [14]

takes inspiration from NGCF and simplifies it by removing its self-connection, feature transformation and linear activation operations. Another line of recommender systems are based on factorization machines. Factorization Machines (FM) combine linear regression with matrix factorization techniques to handle high-order interactions [42]. [2] was also created to mine higher-order interactions. Its deep variants - QNFM [7], DeepFM [13] and xLightFM [17] have been proposed too. Moreover, numerous methods have been proposed to leverage heterogeneous data for recommendation. For instance, [6, 12, 25] have been devised to extract information from user reviews or hashtags. [24, 52] have been proposed to exploit videos and images for recommendation. Location information has also been utilized to assist location-aware recommendations [4].

2.2 Lightweight Embeddings For Recommenders

Various classes of works have investigated learning compressed embeddings to provide a low-memory alternative to traditional embedding tables in recommender systems [63]. For examples, feature selection search methods [32, 36, 54] are trained to learn importance scores, based on which they filter out the unimportant feature fields. Embedding dimension search approaches [22, 34, 35, 40, 41, 62] improve traditional recommender systems with fixed and uniform sizes by introducing mixed dimensions for each feature. Quantization-based methods [5, 9, 10, 19, 21, 26] reduces the precision of numerical values and store the parameters with lower bits. Feature hashing approaches [1, 11, 28, 30, 33, 46, 47, 57, 60] utilize various hash tricks to transform input values to a smaller range, thus compression the embedding table. Knowledge distillation techniques train a smaller student model from a larger teacher model and have been used to learn an embedding table of smaller sizes from larger ones [55, 56]. BET achieves dynamic embedding sizes and aims to find the optimal embedding size for each feature, so it should be classified as a embedding dimension search approach.

3 PROPOSED APPROACH

In a nutshell, BET has three components: (1) a pre-trained backbone recommender with a full-size embedding table $G_\Theta(\cdot)$ parameterized by Θ ; (2) an action fitness prediction network $F_\Phi(\cdot)$ parameterized by Φ ; and (3) a non-parametric action sampler $H(\cdot)$. Figure 1 provides an overview of the workflow of BET. Under pre-specified memory budgets, $H(\cdot)$ samples actions from a set of probabilistic distributions in each iteration. Each action specifies the embedding sizes for all users and items. Afterwards, $F_\Phi(\cdot)$ estimates the fitness of each action and selects the best action predicted. $G_\Theta(\cdot)$ is further finetuned by adopting the embedding sizes in the chosen action, and is then evaluated on a hold-out dataset to obtain the actual fitness score (i.e., recommendation quality) of the action. This facilitates the optimization of $F_\Phi(\cdot)$ by minimizing its prediction error, thus leading us to optimal actions under each given memory budget after T iterations. In the remainder of this section, we unfold the design of each component.

3.1 Recommender with Embedding Masking

Let $\mathcal{U}$ and $\mathcal{V}$ denote a set of users u and items v , respectively. The embedding table in the recommender maps each integer-valued user

or item IDs to a real-valued embedding vector $\mathbf{e}_n$, where $n \in \mathcal{U} \cup \mathcal{V}$ indexes either a user or an item. Note that we only separately use u and v to index users and items when they need to be distinguished, e.g., when performing pairwise ranking. The vertical concatenation of all embedding vectors forms the embedding table $\mathbf{E}$ of size $(|\mathcal{U}| + |\mathcal{V}|) \times d_{max}$, where d_{max} is the maximum/full embedding size supported by $G_\Theta(\cdot)$. The embedding sizes are adjusted via a binary mask $\mathbf{M} \in \{0, 1\}^{(|\mathcal{U}| + |\mathcal{V}|) \times d_{max}}$, which is commonly adopted for embedding sparsification in lightweight recommender systems [35, 37, 41]. Each row $\mathbf{m}_n$ in $\mathbf{M}$ is a mask vector corresponding to user/item n . Given a searched embedding size d_n , we set the first d_n elements of $\mathbf{m}_n$ to ones the remaining $d_{max} - d_n$ dimensions to zeros. Then, we perform element-wise multiplication of the embedding table $\mathbf{E}$ and the binary mask matrix $\mathbf{M}$ to adjust the embedding sizes during embedding look-up, i.e., $\mathbf{e}_n = (\mathbf{E} \odot \mathbf{M})[n]$ where $[n]$ retrieves the n -th row of a matrix. $\mathbf{M}$ is adaptively updated based on the sampled actions in every iteration. It is worth noting that, in the deployment stage, only the heavily sparsified embedding table $\mathbf{E}_{sparse} = \mathbf{E} \odot \mathbf{M}$ will be stored, where the cost of storing zero-valued entries can be negligible with sparse matrix storage techniques [44, 49]. Once we obtain the sparse embedding vectors, the recommender model $G_\Phi(\cdot)$ computes a preference score $\hat{y}_{uv}$ representing pairwise user-item similarity:

$$\hat{y}_{uv} = G_\Theta(\mathbf{e}_u, \mathbf{e}_v), \quad u \in \mathcal{U}, v \in \mathcal{V}, \quad (1)$$

where $G_\Theta(\cdot)$ can be any latent factor-based recommender model that supports pairwise scoring.

For optimizing the recommender, we adopt Bayesian Personalized Ranking (BPR) loss [43]:

$$\mathcal{L}_{BPR} = \sum_{(u,v,v') \in \mathcal{D}_{train}} -\ln \sigma(\hat{y}_{uv} - \hat{y}_{uv'}) + \eta \|\Theta\|_2^2, \quad (2)$$

where $(u, v, v') \in \mathcal{D}_{train}$ represents the training samples where user u chooses item v over item v' , while $\hat{y}_{uv}$ and $\hat{y}_{uv'}$ respectively represent the predicted preferences that user u has on items v and v' . $\eta \|\Theta\|_2^2$ provides regularization to prevent overfitting, with a scaling factor η . Our goal is to search for the optimal embedding sizes for all users and items under strict memory budgets, hence the overall objective is defined as follows:

$$\min_{\Theta, \Phi} \mathcal{L}_{BPR} \quad \text{s.t.} \quad 1 - \frac{\|\mathbf{M}\|_{1,1}}{(|\mathcal{U}| + |\mathcal{V}|) \times d_{max}} \leq c, \quad (3)$$

where $\|\mathbf{M}\|_{1,1}$ counts the number of ones, or equivalently retained parameters in the binary mask, c ($0 < c < 1$) is the target sparsity rate, i.e., the ratio between the numbers of active parameters in sparsified and full embedding tables.

3.2 Budget-Aware Action Sampling

Hereby we introduce a novel table-level action sampling strategy. In each search iteration, we randomly sample a set of candidate actions $\mathcal{Q} = \{a_1, \dots, a_m\}$. Each action assigns a continuous embedding size $d_n \in \mathbb{N}_{\leq d_{max}}^+$ to every user or item $n \in \mathcal{U} \cup \mathcal{V}$.

As we aim to avoid optimizing the selected action towards an implicit, manually tuned memory cost term, we propose to directly draw a table-level action that: (1) makes all generated embedding sizes conditioned on a certain distribution; and (2) ensures the total parameter size of all embeddings is capped at the budgeted

parameter consumption. In short, (1) essentially narrows the action space to mitigate the difficulty of searching from a huge collection of completely random actions, while (2) guarantees that the table-level action strictly meets the memory budget in the first place.

To generate an action a , we first determine two probability distributions $P_{\mathcal{U}}$ and $P_{\mathcal{V}}$ respectively for users and items. Meanwhile, to introduce inductive bias to the action sampler, instead of resorting to a fixed distribution throughout the search process, $P_{\mathcal{U}}$ and $P_{\mathcal{V}}$ are both randomly chosen from a set of candidate distributions with on-the-fly parameterization. Specifically, we utilize a power law distribution, a truncated exponential distribution, a truncated normal distribution and a log normal distribution. For simplicity, we let the two selected distributions respectively be parameterized by $\beta_{\mathcal{U}}$ and $\beta_{\mathcal{V}}$. In each iteration, the parameters are all determined uniformly at random:

$$\beta_{\mathcal{U}}, \beta_{\mathcal{V}} \sim \text{Uniform}(0, \beta_{\max}), \quad (4)$$

where $\beta_{\max}$ is the maximal value allowed. Notably, the embedding sizes of each field (i.e., users and items) are derived by independently sampling from two distributions $P_{\mathcal{U}}$ and $P_{\mathcal{V}}$. This is because each field may have distinct properties [37] and different distributions are needed to model the embedding sizes of users and items.

After the distributions are determined, we sample $|\mathcal{U}|$ and $|\mathcal{V}|$ probabilities respectively from $P_{\mathcal{U}}$ and $P_{\mathcal{V}}$. The set of probabilities are respectively denoted as $\{p_i\}_{i=1}^{|\mathcal{U}|} \sim P_{\mathcal{U}}$ and $\{p_j\}_{j=1}^{|\mathcal{V}|} \sim P_{\mathcal{V}}$. We then normalize all p_i and p_j as the following:

$$\tilde{p}_i = \frac{p_i}{\sum_i p_i}, \quad \tilde{p}_j = \frac{p_j}{\sum_j p_j}, \quad (5)$$

where $\tilde{p}_i/\tilde{p}_j$ indicates the fraction of parameters each user/item receives among all users/items. Next, we obtain the actual embedding sizes by applying the fraction on the memory budget:

$$\begin{aligned} d_i &= \lfloor \tilde{p}_i \cdot w \cdot d_{\max} \cdot (|\mathcal{U}| + |\mathcal{V}|) \rfloor, \quad i = 1, 2, \dots, |\mathcal{U}|, \\ d_j &= \lfloor \tilde{p}_j \cdot (1 - w) \cdot d_{\max} \cdot (|\mathcal{U}| + |\mathcal{V}|) \rfloor, \quad j = 1, 2, \dots, |\mathcal{V}|, \end{aligned} \quad (6)$$

where w controls the proportion of parameters allocated to the users. In practice, w can be empirically defined, such as an even split (i.e., 0.5). In BET, w is sampled from a uniform distribution each time a table-level action is generated, i.e., $w \sim \text{Uniform}(0, 1)$, so as to probe more embedding size allocation strategies in the iterative search process.

With $|\mathcal{U}| + |\mathcal{V}|$ embedding sizes derived, we now allocate them to each user and item. Previous studies [37, 48, 62] have shown that more frequent users and items tend to be more informative than long-tail ones, hence generally benefit from a larger embedding dimension which allows more information to be encoded. Therefore, we rank the users by their frequency and allocate the n -th largest user embedding size to the n -th frequent user. Similarly, we assign embedding sizes to items based on their frequency. As such, we are capable of sampling m random table-level actions in each search iteration to facilitate the subsequent steps.

3.3 Fitness Prediction Network

In each iteration t we sample a set of m actions, denoted as $\mathcal{Q}_t$. To select the optimal action from $\mathcal{Q}_t$, a straightforward approach is to pair each sampled action with a fresh recommender, train it until

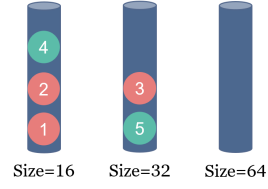


Figure 2: Example of our set-based action formulation. Suppose we have three legitimate embedding sizes $\{16, 32, 64\}$, four users (red) and two items (green). Then, size 16 is a set containing user 1, user 2 and item 4, which means their embedding sizes are 16. Similarly, the action specifies the embedding sizes of user 3 and item 5 to be 32. Size 64 is not used by any users/items.

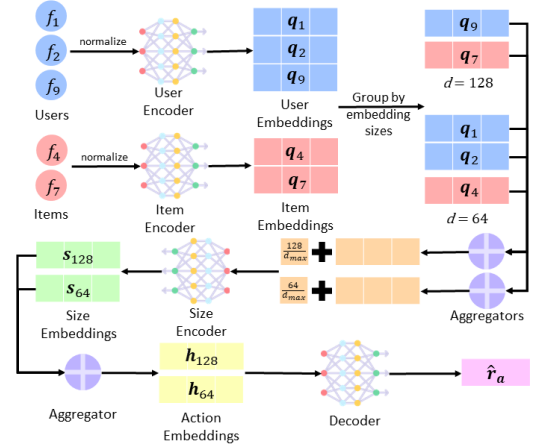


Figure 3: Overview of the DeepSets-based fitness predictor.

convergence and select the action that yields the best final recommendation quality on the validation set. However, this approach faces inevitable computational burdens given the m training and evaluation processes needed per search iteration.

In BET, we utilize an action fitness predictor, which inductively learns distinct embeddings of actions and predicts its fitness score (i.e., recommendation quality) based on the action embeddings. As discussed earlier, learning expressive representations of different actions is a crucial yet non-trivial task for effective fitness prediction, especially considering the need for inductively composing embeddings for unseen actions in every new search iteration.

Thus, we propose a novel action fitness prediction network based on the notion of set representation learning [50, 59]. Before delving into the fitness prediction network, we first detail the set-based formulation of each action. Each action is regarded as a collection of sets $a = \{\mathcal{S}_d\}_{d=1}^{d_{\max}}$, where each set $\mathcal{S}_d = \{n | \forall n \in \mathcal{U} \cup \mathcal{V}, d_n = d\}$ stores the users and items that use the embedding size $d \in [1, d_{\max}]$. $\mathcal{S}_d = \emptyset$ if there is no user or item assigned to embedding size d . A toy example with is given in figure 2. As such, this allows us to build our fitness predictor upon DeepSets [59] owing to its ability to learn permutation-variant representations of sets.

The fitness prediction network consists of four components: a user encoder $\rho_{\mathcal{U}}(\cdot)$, an item encoder $\rho_{\mathcal{V}}(\cdot)$, an embedding size encoder $\mu(\cdot)$, and a decoder $\pi(\cdot)$, all of which bear a distinct MLP. Encoders $\rho_{\mathcal{U}}(\cdot)/\rho_{\mathcal{V}}(\cdot)$ take some contextual information about a

user/item, and produce their corresponding representations. Previous studies [41, 62] have demonstrated the suitability of users'/items' popularity and current embedding sizes as the context information for embedding size search. We inherit this design by establishing a hierarchical, set-compatible action representation learning scheme. Specifically, $\rho_{\mathcal{U}}(\cdot)$ and $\rho_{\mathcal{V}}(\cdot)$ respectively takes the user and item frequencies to generate frequency-weighted embeddings:

$$\begin{aligned} \mathbf{q}_u &= \rho_{\mathcal{U}}\left(\frac{f_u}{\max_{u' \in \mathcal{U}} f_{u'}}\right), \quad u \in \mathcal{U}, \\ \mathbf{q}_v &= \rho_{\mathcal{V}}\left(\frac{f_v}{\max_{v' \in \mathcal{V}} f_{v'}}\right), \quad v \in \mathcal{V}, \end{aligned} \quad (7)$$

where each f_u/f_v is normalized by the maximum user/item frequency observed.

As mentioned above, within an action a , each set $\mathcal{S}_d$ contains all users and items with embedding size d . To obtain embedding $\mathbf{s}_d$ of set $\mathcal{S}_d$, we firstly fuse the representations of all involved users and items using mean aggregation, and then append the explicit embedding size information:

$$\mathbf{s}_d = \mu\left(\left[\frac{1}{|\mathcal{S}_d|} \sum_{n \in \mathcal{S}_d} \mathbf{q}_n; \frac{d}{d_{\max}}\right]\right), \quad (8)$$

where $[\cdot]$ represents concatenation. With the representations generated for all $d_{\max}$ sets, we derive the embedding for the entire action a by further merging all set representations:

$$\mathbf{h}_a = \frac{1}{d_{\max}} \sum_{d=1}^{d_{\max}} \mathbf{s}_d, \quad (9)$$

where we use $\mathbf{h}_a$ to denote an action embedding. An additional benefit from the set-based action embeddings is that, whenever the action sampler draws any action that is new to the fitness predictor (which is commonly the case in embedding size search), it allows for generating the action embedding in an inductive fashion with the available context information and the updated set formulation.

With each action embedding $\mathbf{h}_a$, the decoder $\pi(\cdot)$ takes $\mathbf{h}_a$ and outputs $\hat{r}_a$, which is the predicted fitness score of action a :

$$\hat{r}_a = \pi(\mathbf{h}_a). \quad (10)$$

We will detail how the entire fitness prediction network $F_{\Phi}(\cdot)$ is optimized along with the search process in the next section.

3.4 Fitness Predictor Training

Though bearing a relatively simple structure, the fitness predictor still needs to be trained with some ground truth samples. The fitness prediction network $F_{\Phi}(\cdot)$ is optimized by minimizing the mean squared error between the predicted fitness score $\hat{r}_a$ and the actual one r_a with gradient descent:

$$\Phi = \operatorname{argmin}_{\Phi'} \sum_a (r_a - F_{\Phi'}(a))^2 = \operatorname{argmin}_{\Phi'} \sum_a (r_a - \hat{r}_a)^2. \quad (11)$$

The actual fitness score r_a , i.e., recommendation quality of the action, can be evaluated on the backbone recommender $G_{\Theta}(\cdot)$ after being finetuned with an action a w.r.t. Eq. 2:

$$r_a = \frac{\operatorname{eval}(\mathbf{E} \odot \mathbf{M}_a | \mathcal{D}_{\text{val}})}{\operatorname{eval}(\mathbf{E} | \mathcal{D}_{\text{val}})}, \quad (12)$$

Algorithm 1 Embedding Size Search with BET

```

1: Initialize  $\Theta$  and  $\Phi$ , set  $\mathcal{A} \leftarrow \emptyset$ ;
2: Train  $G_{\Theta}(\cdot)$  till convergence w.r.t. Eq. 2;
3: for  $t = 1, \dots, T$  do
4:   Draw distributions and their parameters w.r.t. Eq. 4;
5:   Draw  $m$  actions  $\mathbf{Q}_t = \{a_1, \dots, a_m\}$  with Eq. 5 and Eq. 6;
6:   if  $0 \leq t \bmod 5 \leq 2$  then
7:      $a_t \leftarrow$  Strategy I;
8:   else if  $t \bmod 5 = 3$  then
9:      $a_t \leftarrow$  Strategy II;
10:  else if  $t \bmod 5 = 4$  then
11:     $a_t \leftarrow$  Strategy III;
12:  end if
13:  Finetune  $\Theta$  w.r.t.  $\mathbf{E} \odot \mathbf{M}_{a_t}$  and Eq. 2
14:   $r_{a_t} \leftarrow$  Eq. 13
15:   $\mathcal{A} \leftarrow \mathcal{A} \cup (a_t, r_{a_t})$ ;
16:  for  $j = 1, \dots, N$  do
17:    Randomly sample  $(a_j, r_{a_j})$  from  $\mathcal{A}$ ;
18:     $\hat{r}_{a_j} \leftarrow$  Eq. 10;
19:    Update  $\Phi$  w.r.t. Eq. 11;
20:  end for
21: end for
22: Perform selective retraining and obtain  $a^* \leftarrow$  Eq. 14.

```

where $\operatorname{eval}(\cdot)$ evaluates the recommendation quality, $\mathcal{D}_{\text{val}}$ is the validation set, $\mathbf{M}_a$ is the binary mask corresponding to action a , and the denominator is the recommendation quality of the pre-trained backbone recommender $G_{\Theta}(\cdot)$ with the full embedding table. Hence, r_a can be interpreted as the ratio between the two recommendation accuracy obtained by the sparsified embeddings and the full embeddings. The implementation of $\operatorname{eval}(\cdot)$ can follow most commonly used recommendation metrics like Recall@ k and NDCG@ k [14, 43], where we adopt an ensemble of both metrics under different values of k :

$$\operatorname{eval}(\mathbf{E} \odot \mathbf{M} | \mathcal{D}_{\text{val}}) = \sum_{u \in \mathcal{U}} \frac{\sum_{k \in \mathcal{K}} \operatorname{Recall}@k_u + \operatorname{NDCG}@k_u}{2|\mathcal{K}||\mathcal{U}|}, \quad (13)$$

where we the choices of k cover $\mathcal{K} = \{5, 10, 20\}$ in our paper.

The ground truth samples can be thus generated by gathering different action-fitness pairs. Instead of constructing a pool of such training samples and train $F_{\Phi}(\cdot)$ upfront, we incrementally add one training sample in every search iteration t to optimize $F_{\Phi}(\cdot)$ on the go. The training sample is constructed by firstly identifying a promising action from $\mathbf{Q}_t$, denoted by a_t , and then obtaining r_{a_t} by evaluating a_t on the finetuned $G_{\Theta}(\cdot)$. Compared with most RL-based embedding size search methods, the required number of fitness evaluation with the recommendation model will be reduced from $m \times T$ to merely T , where T is the total number of iterations as well as the ground truth samples for optimizing $F_{\Phi}(\cdot)$.

3.5 Action Selection and Embedding Retraining

The pseudocode for BET is presented in Algorithm 1. Among its T search iterations, we utilize three different strategies to pick one action a_t from the candidate action set $\mathbf{Q}_t$ (lines 4-12). Then, the recommender $G_{\Theta}(\cdot)$ is finetuned by adopting a_t and evaluated on the validation set (lines 13-14), where the produced (a_t, r_{a_t}) sample

is popped into the population set $\mathcal{A}$ to facilitate training the fitness prediction network $F_\Phi(\cdot)$ (lines 16-20). Finally, a selective retraining process is conducted with the top-ranked actions in the population in order to identify the best embedding size decision for all users and items. We provide further details about action selection strategies and retraining the sparsified embeddings below.

3.5.1 Diversifying the Action Space. The iterative input embedding size search contains T iterations. In each iteration t , the selected action a_t is added to the population set $\mathcal{A}$ with its associated fitness score. For efficiency consideration, in each iteration t , we only compute the actual fitness score for action a_t , which is selected from candidate actions Q_t with one of the three following strategies:

- I: We use our fitness prediction network $F_\Phi(\cdot)$ to predict the fitness score of each candidate actions, and greedily selects the action predicted to have the highest fitness score, i.e., $a_t = \operatorname{argmax}_{a \in Q_t} F_\Phi(a)$. In essence, $F_\Phi(\cdot)$ is a surrogate function that is trained to approximate Eq. 12 and is substantially more cost-effective than evaluating all m actions via recommender finetuning.
- II: We randomly select one action from the Q_t to diversify the training samples for $F_\Phi(\cdot)$ by preventing it from overfitting some specific action distributions in the population $\mathcal{A}$.
- III: We identify currently the best action (with the highest r_a) in $\mathcal{A}$. All actions in Q_t and the best action are inductively embedded into vector representations using Eq. 9. Then, by calculating the pairwise Euclidean distance, we select action $a_t \in Q_t$ by locating the best action's nearest neighbor in the embedding space.

3.5.2 Selective Retraining for Sparsified Embeddings. After T search iterations, from the population we identify the top five actions with the highest r_a on the validation set, denoted as $\mathcal{A}'$. Each action $a \in \mathcal{A}'$ is paired with a fresh backbone recommender model, which is trained from scratch on the training set until convergence. Next, we evaluate the trained recommenders and select the final action a^* that yields the best validation performance:

$$a^* = \operatorname{argmax}_{a \in \mathcal{A}'} r_a, \quad (14)$$

with which the sparsified embedding $\mathbf{E}_{\text{sparse}}$ is obtained and stored for inference.

4 EXPERIMENTS

This section details the experimental results of BET's performance.

4.1 Base Recommenders and Comparative Methods

One can combine BET with different embedding-based representation learning backbone recommender models. We tested the effectiveness of our approach with three commonly used recommender models as the backbone recommender F_Φ : NCF [15], NGCF [53] and LightGCN [14]. We used the same settings as reported in their original works and replaced the embedding table with one with adjustable embedding sizes to demonstrate the versatility and generalizability of our method across various backbone recommenders. We compare BET with the following model-agnostic embedding size search algorithms:

- PEP [35]: It adaptively prunes the embedding parameters based on a learned pruning threshold learned along with the model's other parameters during the training process.
- ESAPN [?]: It uses an RL agent which performs hard selection on embedding sizes for users and items based on the memory complexity and recommendation performance.
- OptEmbed [37]: It prunes redundant embeddings based on learnable pruning thresholds indicative of features' importance and derives the optimal embedding size for each field using evolutionary search.
- CIESS [41]: It is an RL-based method that chooses embedding sizes for the users and items from a continuous domain.
- Single and Uniform (SU): All users and items share equal and fixed embedding sizes.
- Single and Random (SR): The embedding sizes of the users and items are sampled from a uniform distribution.

4.2 Evaluation Protocols

All the aforementioned methods are evaluated on two real-world datasets: Gowalla [29] and Yelp2018 [53]. The first has 1,027,370 interactions between 29,858 users and 40,981 items while the latter has 1,561,147 interactions between 31,668 users and 38,048 items. We take 50%, 25% and 25% of the two datasets for training, validation, and test. To measure the recommendation performance of the backbone recommender, Recall@ k and NDCG@ k are used as evaluation metrics, where k is set to either 5 or 20. To test the recommendation performance of CIESS, PEP, SU and SR, three different sparsity ratios (c) of 80%, 90% and 95% are used. For each method, the compressed embedding table is ensured to have no more than $c \cdot d_{\max} \cdot (|\mathcal{U}| + |\mathcal{V}|)$ usable parameter and the maximal embedding size is $d_{\max} = 128$. However, since ESAPN and OptEmbed are not designed to precisely control the embedding sparsity and focus more on the performance, this section only reports the performance associated with their final embedding tables after pruning.

4.3 Implementation Notes for BET

Hereby we detail the implementation of BET. The number of search iterations T is 10. The maximal values of the distribution parameters $\beta_{\max}$ are set to 20 for power law distribution, 20 for truncated normal distribution, 5 for truncated exponential distribution and 0.5 for log normal distribution. The maximal embedding size $d_{\max}$ is 128 and the minimal embedding size $d_{\min}$ is 1. In each iteration, the pre-trained backbone recommender is finetuned for 10 epochs (i.e., $h = 10$) when the backbone recommender is LightGCN or NCF. When NGCF is used, the backbone recommender is finetuned for 15 epochs (i.e., $h = 15$). For finetuning, the initial learning rate is 0.03 and decays every 200 steps with a decay ratio of 0.98. The number of fitness prediction updates in each iteration, or N , is set to 2. The user and item encoders have input size 1, output size 16, and hidden size 16. The size encoder has input size 17, output size 64, and hidden size 64. The decoder has input size 64, output size 1, and hidden size 64. All the MLP-based encoders and decoders have two layers with the Leaky ReLU [38] activation function.

LightGCN						NGCF					NCF				
Method	Sparsity	R@5	N@5	R@20	N@20	Sparsity	R@5	N@5	R@20	N@20	Sparsity	R@5	N@5	R@20	N@20
ESAPN	75%	0.0741	0.1173	0.1579	0.1329	83%	0.0627	0.1060	0.1396	0.1193	74%	0.0498	0.0718	0.1280	0.0959
OptEmb	74%	0.0591	0.0983	0.1314	0.1115	75%	0.0510	0.0860	0.1227	0.1005	82%	0.0330	0.0492	0.0877	0.0653
PEP	80%	0.0715	0.1189	0.1588	0.1347	80%	0.0656	0.1049	0.1524	0.1245	80%	0.0498	0.0704	0.1282	0.0957
SU		0.0756	0.1219	0.1674	0.1400		0.0619	0.0961	0.1466	0.1170		0.0481	0.0662	0.1235	0.0916
SR		0.0708	0.1150	0.1583	0.1323		0.0512	0.0815	0.1223	0.0982		0.0478	0.0656	0.1243	0.0910
CIESS		0.0777	0.1263	0.1750	0.1461		0.0676	0.1084	0.1504	0.1251		0.0550	0.0744	0.1307	0.0997
BET		0.0797	0.1303	0.1753	0.1483		0.0643	0.1039	0.1454	0.1204		0.0511	0.0723	0.1294	0.0973
<i>p</i> -value		0.0024	0.0001	0.0391	0.0012		0.7815	0.2179	0.0514	0.0924		0.1962	0.1337	0.2578	0.0414
PEP	90%	0.0594	0.0992	0.1274	0.1102	90%	0.0552	0.0914	0.1327	0.1081	90%	0.0414	0.0631	0.1121	0.0839
SU		0.0646	0.1041	0.1477	0.1217		0.0589	0.0935	0.1380	0.1115		0.0413	0.0548	0.1112	0.0796
SR		0.0617	0.1013	0.1403	0.1171		0.0466	0.0764	0.1123	0.0911		0.0399	0.0536	0.1069	0.0769
CIESS		0.0721	0.1193	0.1589	0.1353		0.0601	0.0980	0.1380	0.1140		0.0448	0.0613	0.1131	0.0841
BET		0.0736	0.1212	0.1620	0.1376		0.0623	0.1012	0.1426	0.1177		0.0434	0.0648	0.1122	0.0845
<i>p</i> -value		0.0165	0.0481	0.0091	0.0226		0.0005	0.0205	0.0601	0.0232		0.1454	0.0554	0.6608	0.0655
PEP	95%	0.0372	0.0666	0.0803	0.0712	95%	0.0247	0.0471	0.0674	0.0554	95%	0.0351	0.0591	0.0897	0.0712
SU		0.0446	0.0727	0.1085	0.0872		0.0527	0.0870	0.1213	0.1007		0.0336	0.0478	0.0952	0.0682
SR		0.0500	0.0834	0.1178	0.0975		0.0438	0.0739	0.1040	0.0859		0.0329	0.0489	0.0918	0.0674
CIESS		0.0513	0.0853	0.1214	0.0997		0.0505	0.0856	0.1129	0.0964		0.0361	0.0567	0.0933	0.0717
BET		0.0627	0.1037	0.1374	0.1171		0.0528	0.0883	0.1172	0.0980		0.0388	0.0616	0.0963	0.0756
<i>p</i> -value		0.0001	0.0002	0.0001	0.0001		0.1896	0.0638	0.7269	0.3608		0.0321	0.1107	0.1104	0.0010

(a) Results on Gowalla

LightGCN						NGCF					NCF				
Method	Sparsity	R@5	N@5	R@20	N@20	Sparsity	R@5	N@5	R@20	N@20	Sparsity	R@5	N@5	R@20	N@20
ESAPN	73%	0.0294	0.0696	0.0812	0.0773	71%	0.0214	0.0538	0.0632	0.0603	76%	0.0153	0.0342	0.0490	0.0430
OptEmb	80%	0.0183	0.0424	0.0534	0.0489	67%	0.0133	0.0301	0.0412	0.0369	77%	0.0076	0.0161	0.0238	0.0203
PEP	80%	0.0253	0.0605	0.0723	0.0682	80%	0.0086	0.0167	0.0275	0.0223	80%	0.0130	0.0236	0.0406	0.0327
SU		0.0289	0.0665	0.0822	0.0758		0.0223	0.0517	0.0665	0.0603		0.0131	0.0226	0.0465	0.0351
SR		0.0253	0.0588	0.0783	0.0698		0.0212	0.0499	0.0618	0.0573		0.0101	0.0183	0.0342	0.0268
CIESS		0.0292	0.0692	0.0839	0.0783		0.0233	0.0566	0.0701	0.0652		0.0175	0.0377	0.0533	0.0474
BET		0.0310	0.0726	0.0873	0.0816		0.0254	0.0601	0.0741	0.0685		0.0194	0.0451	0.0620	0.0554
<i>p</i> -value		0.0008	0.0309	0.0004	0.0109		0.0412	0.0418	0.0082	0.0024		0.0387	0.0122	0.0483	0.0341
PEP	90%	0.0224	0.0531	0.0657	0.0610	90%	0.0080	0.0153	0.0276	0.0215	90%	0.0139	0.0242	0.0427	0.0335
SU		0.0230	0.0544	0.0722	0.0648		0.0205	0.0528	0.0622	0.0592		0.0119	0.0201	0.0416	0.0309
SR		0.0210	0.0476	0.0642	0.0573		0.0195	0.0449	0.0573	0.0524		0.0089	0.0170	0.0309	0.0242
CIESS		0.0263	0.0649	0.0730	0.0705		0.0232	0.0551	0.0669	0.0619		0.0153	0.0350	0.0500	0.0442
BET		0.0284	0.0674	0.0816	0.0762		0.0246	0.0586	0.0736	0.0676		0.0173	0.0400	0.0565	0.0499
<i>p</i> -value		0.0045	0.0060	0.0051	0.0006		0.0012	0.0007	0.0102	0.0069		0.0234	0.0067	0.0426	0.0118
PEP	95%	0.0199	0.0496	0.0600	0.0561	95%	0.0075	0.0143	0.0259	0.0206	95%	0.0125	0.0244	0.0399	0.0330
SU		0.0217	0.0494	0.0624	0.0571		0.0196	0.0496	0.0573	0.0545		0.0091	0.0174	0.0303	0.0238
SR		0.0195	0.0465	0.0583	0.0540		0.0175	0.0415	0.0528	0.0486		0.0078	0.0145	0.0275	0.0216
CIESS		0.0230	0.0534	0.0657	0.0640		0.0222	0.0540	0.0662	0.0613		0.0157	0.0383	0.0483	0.0446
BET		0.0255	0.0596	0.0727	0.0676		0.0216	0.0505	0.0639	0.0588		0.0170	0.0404	0.0545	0.0491
<i>p</i> -value		0.0403	0.0200	0.0305	0.0204		0.0327	0.0451	0.0064	0.0193		0.0399	0.0246	0.0094	0.0044

(b) Results on Yelp

Table 1: Performance of all methods on (a) Gowalla, and (b) Yelp2018 with the sparsity ratio set to 80%, 90%, and 95%. The best results are highlighted. Recall@ k and NDCG@ k are denoted by R@ k and N@ k for simplicity. Each p -value is calculated via the paired t -test between BET and the best baseline.

Sparsity	Selection Method	Yelp2018		Gowalla	
		R@20	N@20	R@20	N@20
80%	Random Selection	0.0846	0.0788	0.1749	0.1473
	Simple fitness prediction	0.0835	0.0779	0.1739	0.1469
	DeepSets fitness prediction	0.0873	0.0816	0.1753	0.1483
90%	Random Selection	0.0767	0.0717	0.1594	0.1357
	Simple fitness prediction	0.0768	0.0717	0.1590	0.1355
	DeepSets fitness prediction	0.0816	0.0762	0.1620	0.1376
95%	Random Selection	0.0637	0.0583	0.1335	0.1143
	Simple fitness prediction	0.0673	0.0629	0.1349	0.1152
	DeepSets fitness prediction	0.0727	0.0676	0.1374	0.1171

Table 2: Performance of BET when different selection methods are used.

4.4 Overall Performance Comparison

Table 1 shows the Recall@ k and NDCG@ k scores of various light-weight embedding methods when they are paired with different backbone recommender models. We also perform paired student's t -test between BET and the best baseline method. Compared with the baselines with adjustable sparsity ratios (i.e., PEP, SU, SR, and CIESS), BET achieves better performance across all three backbone

recommenders and at nearly all specified sparsity levels (80%, 90%, and 95%). The associated p -values are below 5% most of the time so we can reject the null hypothesis that the observed performance changes occurred by chance in most scenarios.

Meanwhile, the sparsity ratios achieved by ESAPN and OptEmb range from 67% and 83%, often failing to meet the minimum target of 80%. Despite keeping a relatively higher number of parameters compared to other methods, their recommendation performance is consistently inferior to BET. For instance, when used with LightGCN on the Gowalla dataset, ESAPN requires more memory complexity (75% sparsity ratio) to achieve a comparable performance compared to BET under an 90% sparsity condition.

In summary, BET demonstrates superior performance while maintaining the same memory usage. Similarly, when comparing performance levels, BET proves to be more efficient in memory utilization. Consequently, these findings highlight the benefits of the ongoing embedding size exploration in BET.

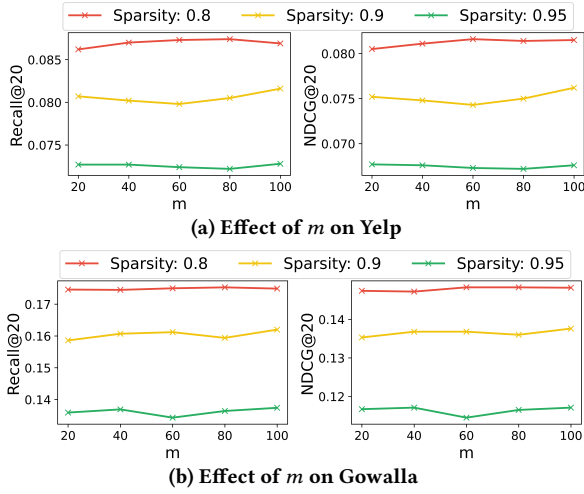


Figure 4: Sensitivity analysis of m on Recall@20 and NDCG@20 with LightGCN as the backbone recommender.

4.5 Model Component Analysis

Selecting from the set of candidate actions in each iteration is a vital step in BET, where we proposed a hybrid selection method that alternately uses three selection strategies. To study the effectiveness of the proposed selection method and the DeepSets-based fitness prediction network, we conduct a quantitative study on BET and its two variants. The first variant uses random selection only. The second variant incorporates a simplistic fitness prediction network that only accepts the embedding sizes of each user and item. Table 2 illustrates the performance of the original version of BET along with its two variants in terms of their Recall@20 and NDCG@20 scores. Only the performance of the best-performing backbone recommender LightGCN is reported due to space limit. The results show that BET with a DeepSets-based fitness prediction network has better performance than the two variants, implying that our proposed selection method is effective and the DeepSets-based architecture is able to learn the latent representation of actions.

4.6 Analysis of Hyperparameter

In this section, we inspect the influence of the core hyperparameters in BET in terms of Recall@20 and NDCG@20. LightGCN shows the best performance so it is chosen as the backbone recommender for demonstration.

4.6.1 Number of Sampled Actions m . In each iteration, we sample m candidate actions to choose from. We tune BET with $m \in \{20, 40, 60, 80, 100\}$ for both Yelp2018 and Gowalla. Then we show how the performance changes with different values of m w.r.t. Recall@20 and NDCG@20 scores. As shown in Figure 4, although BET is relatively insensitive to the value of m , better performance is achieved with $m = 100$ on both datasets.

4.6.2 Number of Iterations T . BET iteratively samples, selects and evaluates actions in search of the optimal one. One natural question that arises is how many iterations it needs to find the action that leads to peak recommendation performance. This question is crucial for understanding the effectiveness of BET. Therefore, we perform selective retraining as described in Section 3.5.2 on the resulting

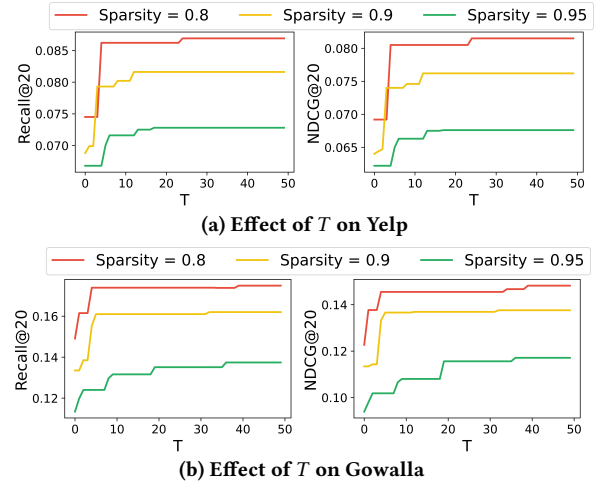


Figure 5: Sensitivity analysis of T on Recall@20 and NDCG@20 with LightGCN as the backbone recommender.

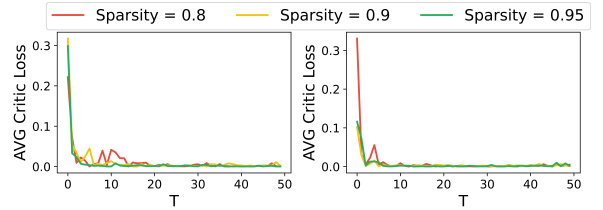


Figure 6: Sensitivity analysis of T on the fitness prediction loss with LightGCN as the backbone recommender on Yelp2018 (left) and Gowalla (right).

population $\mathcal{A}_t$ for each training iteration $t \in [0, T]$ and report the best recommendation performance in terms of Recall@20 and NDCG@20. We also report the average fitness prediction loss in each iteration. Figure 6 shows that fitness predictor converges within the first 15 iterations. While the training loss of the fitness predictor is diminishing, BET witnesses a substantial improvement in its performance in the first 10-15 iterations, as shown in Figure 5. Then performance continues to climb slowly after the 15th iteration until it reaches its peak around the 40th iteration and remains unchanged after that. In conclusion, setting $T = 50$ should suffice for BET to find the optimal action for all sparsity ratios.

5 CONCLUSION AND FUTURE WORKS

Latent factor recommenders typically employ vectorized embeddings with a uniform and fixed size, resulting in suboptimal performance and excessive memory complexity. To address this challenge, we have introduced BET, a model-agnostic algorithm to enable the selection of customized embedding sizes at the level of the embedding table for each user/item. This approach enhances the embeddings while minimizing memory costs. Next, we will develop embedding size search algorithms for streaming recommendation.

ACKNOWLEDGMENT

This work is supported by the Australian Research Council under the streams of Future Fellowship (No. FT210100624), Discovery Project (No. DP190101985), and Discovery Early Career Research Award (No. DE200101465 and No. DE230101033).

REFERENCES

- [1] Anshumali Shrivastava Aditya Desai, Li Chou. 2022. Random Offset Block Embedding (ROBE) for compressed embedding tables in deep learning recommendation systems. In *Proceedings of Machine Learning and Systems*. 762–778.
- [2] Mathieu Blondel, Akinori Fujino, Naonori Ueda, and Masakazu Ishihata. 2016. Higher-Order Factorization Machines. In *Proceedings of the 30th International Conference on Neural Information Processing Systems*. 3359–3367.
- [3] Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. 2020. Once-for-all: Train one network and specialize it for efficient deployment. *ICLR* (2020).
- [4] Cheng Chen, Xiangwu Meng, Zhenghua Xu, and Thomas Lukasiewicz. 2017. Location-Aware Personalized News Recommendation With Deep Semantic Analysis. *IEEE Access* 5 (2017), 1624–1638.
- [5] Ting Chen, Lala Li, and Yizhou Sun. 2020. Differentiable Product Quantization for End-to-End Embedding Compression. In *Proceedings of the 37th International Conference on Machine Learning*. 1617–1626.
- [6] Tong Chen, Hongzhi Yin, Guanhua Ye, Zi Huang, Yang Wang, and Meng Wang. 2020. Try This Instead: Personalized and Interpretable Substitute Recommendation. In *SIGIR*. 891–900.
- [7] Tong Chen, Hongzhi Yin, Xiangliang Zhang, Zi Huang, Yang Wang, and Meng Wang. 2021. Quaternion Factorization Machines: A Lightweight Solution to Intricate Feature Interaction Modeling. *IEEE Transactions on Neural Networks and Learning Systems* 34, 8 (2021), 1–14.
- [8] Tong Chen, Hongzhi Yin, Yujia Zheng, Zi Huang, Yang Wang, and Meng Wang. 2021. Learning Elastic Embeddings for Customizing On-Device Recommenders. In *SIGKDD*. 138–147.
- [9] Yankai Chen, Huifeng Guo, Yingxue Zhang, Chen Ma, Ruiming Tang, Jingjie Li, and Irwin King. 2022. Learning Binarized Graph Representations with Multi-Faceted Quantization Reinforcement for Top-K Recommendation. In *SIGKDD*. 168–178.
- [10] Yankai Chen, Huifeng Guo, Yingxue Zhang, Chen Ma, Ruiming Tang, Jingjie Li, and Irwin King. 2022. Learning Binarized Graph Representations with Multi-Faceted Quantization Reinforcement for Top-K Recommendation. In *KDD*. 168–178.
- [11] Aditya Desai and Anshumali Shrivastava. 2022. The trade-offs of model size in large recommendation models : 100GB to 10MB Criteo-tb DLRM model. In *NeurIPS*. 33961–33972.
- [12] Yuyun Gong and Qi Zhang. 2016. Hashtag recommendation using attention-based convolutional neural network. In *IJCAI*. 2782–2788.
- [13] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *IJCAI* (2017).
- [14] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *SIGIR*. 639–648.
- [15] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *WWW*. 173–182.
- [16] Zhenhua Huang, Xin Xu, Juan Ni, Honghao Zhu, and Cheng Wang. 2019. Multi-modal representation learning for recommendation in Internet of Things. *IEEE Internet of Things Journal* 6, 6 (2019), 10675–10685.
- [17] Gangwei Jiang, Hao Wang, Jin Chen, Haoyu Wang, Defu Lian, and Enhong Chen. 2021. XLightFM: Extremely Memory-Efficient Factorization Machine. In *SIGIR*. 337–346.
- [18] Manas R Joglekar, Cong Li, Mei Chen, Taibai Xu, Xiaoming Wang, Jay K Adams, Pranav Khaitan, Jiahui Liu, and Quoc V Le. 2020. Neural input search for large scale recommendation models. In *SIGKDD*. 2387–2397.
- [19] Wang-Cheng Kang, Derek Zhiyuan Cheng, Ting Chen, Xinyang Yi, Dong Lin, Lichan Hong, and Ed H. Chi. 2020. Learning Multi-Granular Quantized Embeddings for Large-Vocab Categorical Features in Recommender Systems. In *WWW*. 562–566.
- [20] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. 2016. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. In *ICLR*.
- [21] Yunyong Ko, Jae-Seo Yu, Hong-Kyun Bae, Yongjun Park, Dongwon Lee, and Sang-Wook Kim. 2021. MASCOT: A Quantization Framework for Efficient Matrix Factorization in Recommender Systems. In *2021 IEEE International Conference on Data Mining (ICDM)*. 290–299.
- [22] Shuming Kong, Weiye Cheng, Yanyan Shen, and Linpeng Huang. 2023. AutoSrh: An Embedding Dimensionality Search Framework for Tabular Data Prediction. *TKDE* 35, 7 (2023), 6673–6686.
- [23] Su Mon Kywe, Ee-Peng Lim, and Feida Zhu. 2012. A survey of recommender systems in twitter. In *International Conference on Social Informatics*. 420–433.
- [24] Joonseok Lee, Sami Abu-El-Haija, Balakrishnan Varadarajan, and Apostol Natsev. 2018. Collaborative deep metric learning for video understanding. In *Proceedings of the 24th ACM SIGKDD International conference on knowledge discovery & data mining*. 481–490.
- [25] Piji Li, Zihao Wang, Zhaochun Ren, Lidong Bing, and Wai Lam. 2017. Neural rating regression with abstractive tips generation for recommendation. In *Proceedings of the 40th International ACM SIGIR conference on Research and Development in Information Retrieval*. 345–354.
- [26] Shiwei Li, Huifeng Guo, Lu Hou, Wei Zhang, Xing Tang, Ruiming Tang, Rui Zhang, and Ruixuan Li. 2023. Adaptive Low-Precision Training for Embeddings in Click-Through Rate Prediction. *Proceedings of the AAAI Conference on Artificial Intelligence* 37, 4 (2023), 4435–4443.
- [27] Siyuan Li and Elena Karahanna. 2015. Online Recommendation Systems in a B2C E-Commerce Context: A Review and Future Directions. *Journal of the Association for Information Systems* 16 (2015), 72–107.
- [28] Yang Li, Tong Chen, Peng-Fei Zhang, and Hongzhi Yin. 2021. Lightweight self-attentive sequential recommendation. In *CIKM*. 967–977.
- [29] Dawen Liang, Laurent Charlin, James McInerney, and David M. Blei. 2016. Modeling User Exposure in Recommendation. In *WWW*. 951–961.
- [30] Xurong Liang, Tong Chen, Quoc Viet Hung Nguyen, Jianxin Li, and Hongzhi Yin. 2023. Learning Compact Compositional Embeddings via Regularized Pruning for Recommendation. In *ICDM*.
- [31] Guanyu Lin, Chen Gao, Yinfeng Li, Y. Zheng, Zhiheng Li, Depeng Jin, and Yong Li. 2022. Dual Contrastive Network for Sequential Recommendation with User and Item-Centric Perspectives. *ACM Transactions on Information Systems* 1, 1 (2022).
- [32] Weilin Lin, Xiangyu Zhao, Yejing Wang, Tong Xu, and Xian Wu. 2022. AdaFS: Adaptive Feature Selection in Deep Recommender System. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3309–3317.
- [33] Han Liu, Yinwei Wei, Jianhua Yin, and Liqiang Nie. 2023. HS-GCN: Hamming Spatial Graph Convolutional Networks for Recommendation. *TKDE* 35, 6 (2023), 5977–5990.
- [34] Haochen Liu, Xiangyu Zhao, Chong Wang, Xiaobing Liu, and Jiliang Tang. 2020. Automated embedding size search in deep recommender systems. In *SIGIR*. 2307–2316.
- [35] Siyi Liu, Chen Gao, Yihong Chen, Depeng Jin, and Yong Li. 2021. Learnable Embedding sizes for Recommender Systems. In *ICLR*.
- [36] Yuanfei Luo, Mengshuo Wang, Hao Zhou, Quanming Yao, Wei-Wei Tu, Yuqiang Chen, Wenyuan Dai, and Qiang Yang. 2019. Autocross: Automatic feature crossing for tabular data in real-world applications. In *SIGKDD*. 1936–1945.
- [37] Fuyuan Lyu, Xing Tang, Hong Zhu, Huifeng Guo, Yingxue Zhang, Ruiming Tang, and Xue Liu. 2022. OptEmbed: Learning Optimal Embedding Table for Click-through Rate Prediction. In *CIKM*. 1399–1409.
- [38] Andrew L Maas, Awni Y Hannun, Andrew Y Ng, et al. 2013. Rectifier nonlinearities improve neural network acoustic models. In *ICML*.
- [39] Khalil Muhammad, Qinqin Wang, Diarmuid O'Reilly-Morgan, Elias Tragos, Barry Smyth, Neil Hurley, James Geraci, and Aonghus Lawlor. 2020. Fedfast: Going beyond average for faster training of federated recommender systems. In *SIGKDD*. 1234–1242.
- [40] Liang Qu, Yonghong Ye, Ningzhi Tang, Lixin Zhang, Yuhui Shi, and Hongzhi Yin. 2022. Single-Shot Embedding Dimension Search in Recommender System. In *SIGIR*. 513–522.
- [41] Yunke Qu, Tong Chen, Xiangyu Zhao, Lizhen Cui, Kai Zheng, and Hongzhi Yin. 2023. Continuous Input Embedding Size Search For Recommender Systems. In *SIGIR*. 708–717.
- [42] Steffen Rendle. 2010. Factorization machines. In *2010 IEEE International conference on data mining*. 995–1000.
- [43] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*. 452–461.
- [44] Naser Sedaghati, Te Mu, Louis-Noel Pouchet, Srinivasan Parthasarathy, and P Sadayappan. 2015. Automatic selection of sparse matrix representation on GPUs. In *ACM International Conference on Supercomputing*. 99–108.
- [45] Suvash Sedhain, Aditya Krishna Menon, Scott Sanner, and Lexing Xie. 2015. AutoRec: Autoencoders Meet Collaborative Filtering. In *WWW*. 111–112.
- [46] Geet Sethi, Bilge Acun, Niket Agarwal, Christos Kozyrakis, Caroline Trippel, and Carole-Jean Wu. 2022. RecShard: Statistical Feature-Based Memory Optimization for Industry-Scale Neural Recommendation. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 344–358.
- [47] Hao-Jun Michael Shi, Dheevatsa Mudigere, Maxim Naumov, and Jiyan Yang. 2020. Compositional embeddings using complementary partitions for memory-efficient recommendation systems. In *SIGKDD*. 165–175.
- [48] Yang Sun, Fajie Yuan, Min Yang, Guoao Wei, Zhou Zhao, and Duo Liu. 2020. A Generic Network Compression Framework for Sequential Recommender Systems. In *SIGIR '20*. 1299–1308.
- [49] Pauli Virtanen et al. 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* 17, 3 (2020), 261–272.
- [50] Edward Wagstaff, Fabian B Fuchs, Martin Engelcke, Michael A Osborne, and Ingmar Posner. 2022. Universal approximation of functions on sets. *JMLR* 23, 1 (2022), 6762–6817.

- [51] Shoujin Wang, Longbing Cao, Yan Wang, Quan Z Sheng, Mehmet A Orgun, and Defu Lian. 2021. A survey on session-based recommender systems. *ACM Computing Surveys (CSUR)* 54, 7 (2021), 1–38.
- [52] Suhan Wang, Yilin Wang, Jiliang Tang, Kai Shu, Suhas Ranganath, and Huan Liu. 2017. What Your Images Reveal: Exploiting Visual Contents for Point-of-Interest Recommendation. In *Proceedings of the 26th International Conference on World Wide Web*. 391–400.
- [53] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *SIGIR*. 165–174.
- [54] Yejing Wang, Xiangyu Zhao, Tong Xu, and Xian Wu. 2022. Autofield: Automating feature selection in deep recommender systems. In *WWW*. 1977–1986.
- [55] Xin Xia, Hongzhi Yin, Junliang Yu, Qinyong Wang, Guandong Xu, and Quoc Viet Hung Nguyen. 2022. On-Device Next-Item Recommendation with Self-Supervised Knowledge Distillation. In *SIGIR*. 546–555.
- [56] Xin Xia, Junliang Yu, Qinyong Wang, Chaoqun Yang, Nguyen Quoc Viet Hung, and Hongzhi Yin. 2023. Efficient On-Device Session-Based Recommendation. *ACM Transactions Information Systems* 41, 4 (2023).
- [57] Bencheng Yan, Pengjie Wang, Jinqian Liu, Wei Lin, Kuang-Chih Lee, Jian Xu, and Bo Zheng. 2021. Binary Code Based Hash Embedding for Web-Scale Applications. In *CIKM*. 3563–3567.
- [58] Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Lizhen Cui, and Quoc Viet Hung Nguyen. 2022. Are Graph Augmentations Necessary? Simple Graph Contrastive Learning for Recommendation. In *SIGIR*. 1294–1303.
- [59] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J Smola. 2017. Deep Sets. In *NeurIPS*. 3394–3404.
- [60] Caojin Zhang, Yicun Liu, Yuanpu Xie, Sofia Ira Ktena, Alykhan Tejani, Akshay Gupta, Pranay Kumar Myana, Deepak Dilipkumar, Suvadip Paul, Ikuhiro Ihara, et al. 2020. Model size reduction using frequency based double hashing for recommender systems. In *Fourteenth ACM Conference on Recommender Systems*. 521–526.
- [61] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys (CSUR)* 52, 1 (2019), 1–38.
- [62] Xiangyu Zhao, Haochen Liu, Wenqi Fan, Hui Liu, Jiliang Tang, Chong Wang, Ming Chen, Xudong Zheng, Xiaobing Liu, and Xiwang Yang. 2021. Autoemb: Automated embedding dimensionality search in streaming recommendations. In *2021 IEEE International Conference on Data Mining (ICDM)*. 896–905.
- [63] Ruiqi Zheng, Liang Qu, Bin Cui, Yuhui Shi, and Hongzhi Yin. 2022. AutoML for Deep Recommender Systems: A Survey. *ACM Transactions on Information Systems* 101 (2022), 1–38.
- [64] Meizi Zhou, Zhuoye Ding, Jiliang Tang, and Dawei Yin. 2018. Micro Behaviors: A New Perspective in E-Commerce Recommender Systems. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*. 727–735.